Design of data encryption system

Name		**Class**	
Date		**Score**	
Examiner			

Contents

1 Introduction

The full adder is a fundamental digital logic circuit used in arithmetic operations. It is capable of adding three binary digits and produces a sum and a carry-out. This experiment focuses on the design, implementation, and simulation of full adders using VHDL on an FPGA platform using Quartus II and ModelSim simulation tools.

2 Experiment Procedure

2.1 Project Setup and Environment Configuration

(1) Launch Quartus II 13.0 software to begin the FPGA design project.

(2) Create a new project through the "New Project Wizard" interface. Configure the project as follows:

- Project name: *fadd4*
- Configure the top-level entity as the testbench module

(3) For simulation-only applications, allow Quartus II to auto-select the default FPGA device.

(4) Set up the ModelSim simulation environment with appropriate libraries and simulation settings.

2.2 Full Adder Design and Implementation

A full adder is a digital circuit that performs the addition of three binary bits: two operands (A and B) and a carry-in (Cin). It produces two outputs: a sum (S) and a carry-out (Cout).

2.2.1 Full Adder Logic

The truth table for a full adder is:

A	B	Cin	S	Cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

2.2.2 VHDL Implementation

The following VHDL code implements a single full adder cell:

```
LIBRARY IEEE;
USE IEEE.std_logic_1164.ALL;
ENTITY fadd IS
    PORT (a,b,ci: IN std_logic;
    co,sum: OUT std_logic
    );
END fadd;

ARCHITECTURE dataflow OF fadd IS
BEGIN
co<=(a and b)or(b and ci)or(a and ci);
sum<=a xor b xor ci;
END dataflow;
```

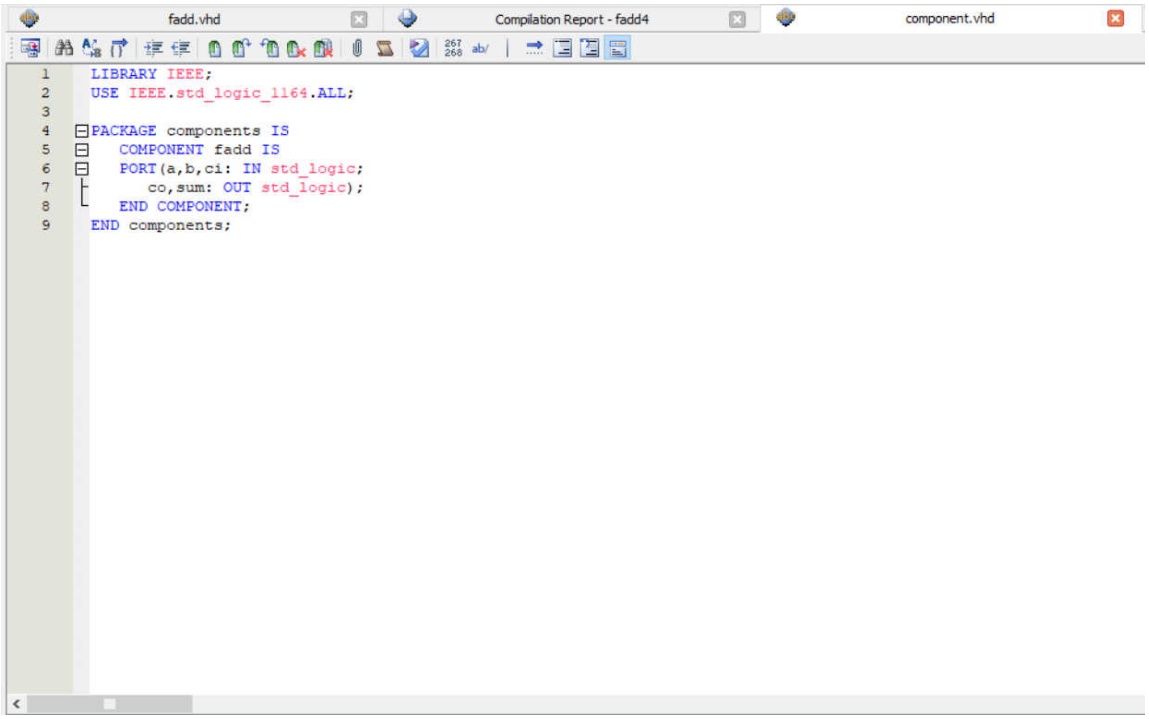

Figure 1: single full adder

2.3 4-Bit Full Adder Design

For multi-bit addition operations, we extend the single full adder to create a 4-bit full adder by cascading four individual full adder cells:

```
LIBRARY IEEE;
USE IEEE.std_logic_1164.ALL;
USE work.components.ALL;

ENTITY fadd4 IS
    PORT (
        a   : IN  std_logic_vector(3 DOWNTO 0);
        b   : IN  std_logic_vector(3 DOWNTO 0);
        ci  : IN  std_logic;
        co  : OUT std_logic;
        sum : OUT std_logic_vector(3 DOWNTO 0)
```

```
    );
END fadd4;

ARCHITECTURE stru OF fadd4 IS
    SIGNAL ci_ns : std_logic_vector(2 DOWNTO 0);
BEGIN
    fa0: fadd
        PORT MAP (
            a   => a(0),
            b   => b(0),
            ci  => ci,
            co  => ci_ns(0),
            sum => sum(0)
        );

    fa1: fadd
        PORT MAP (
            a   => a(1),
            b   => b(1),
            ci  => ci_ns(0),
            co  => ci_ns(1),
            sum => sum(1)
        );

    fa2: fadd
        PORT MAP (
            a   => a(2),
            b   => b(2),
            ci  => ci_ns(1),
            co  => ci_ns(2),
            sum => sum(2)
        );

    fa3: fadd
```

```
        PORT MAP (
            a    => a(3),
            b    => b(3),
            ci   => ci_ns(2),
            co   => co,
            sum  => sum(3)
        );
END stru;
```

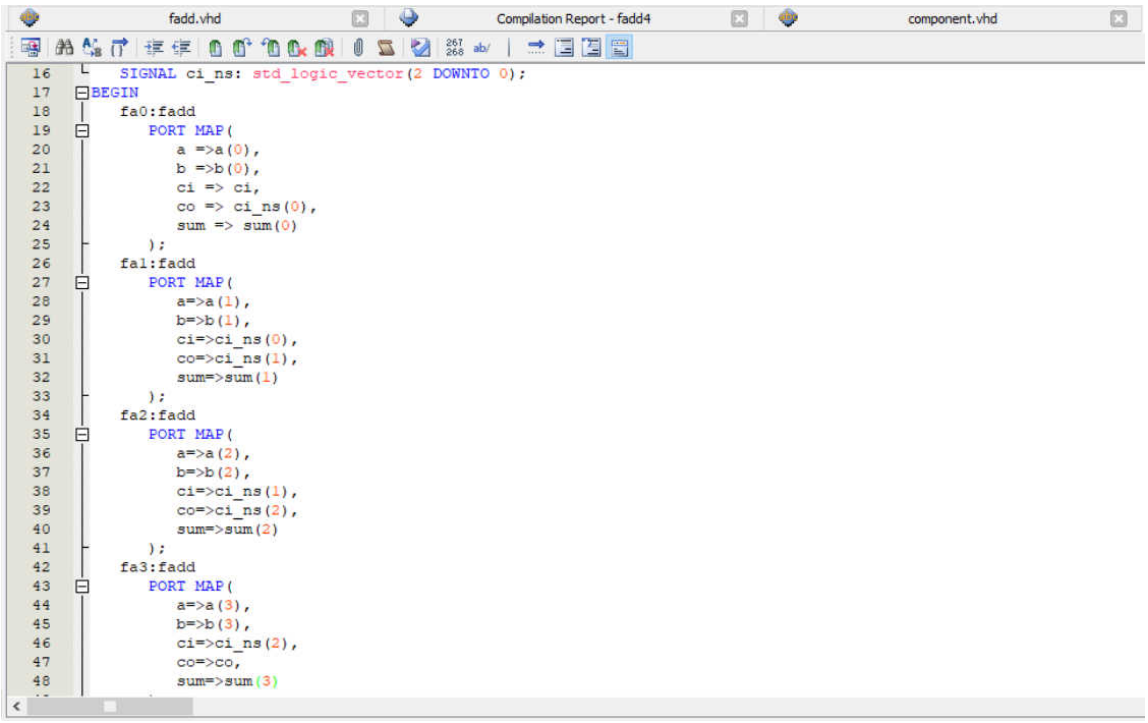

Figure 2: 4-bit full adder

2.4 Testbench Development

At this time, we need to modify it based on the previously generated testbench template, that is, we need to add the following code to the vht file:

```
BEGIN
    -- code that executes only once
```

```
    a<="0000";
    b<="0000";
    ci<='0';
    for k in 0 to 1 loop
        for j in 0 to 3 loop
            for i in 0 to 3 loop
                wait for 10 ns;
                a<=a+1;
            end loop;
            b<=b+1;
        end loop;
        ci<=not ci;
    end loop;
WAIT;
END PROCESS init;
always : PROCESS
```

C:/altera/13.1/fadd4/simulation/modelsim/fadd4.vht (/fadd4_vhd_tst) - Default

```
Ln#
28  USE ieee.std_logic_1164.all;
29  USE ieee.std_logic_unsigned.all;
30
31  ENTITY fadd4_vhd_tst IS
32  END fadd4_vhd_tst;
33  ARCHITECTURE fadd4_arch OF fadd4_vhd_tst IS
34  -- constants
35  -- signals
36  SIGNAL a : STD_LOGIC_VECTOR(3 DOWNTO 0);
37  SIGNAL b : STD_LOGIC_VECTOR(3 DOWNTO 0);
38  SIGNAL ci : STD_LOGIC;
39  SIGNAL co : STD_LOGIC;
40  SIGNAL sum : STD_LOGIC_VECTOR(3 DOWNTO 0);
41  COMPONENT fadd4
42          PORT (
43          a : IN STD_LOGIC_VECTOR(3 DOWNTO 0);
44          b : IN STD_LOGIC_VECTOR(3 DOWNTO 0);
45          ci : IN STD_LOGIC;
46          co : OUT STD_LOGIC;
47          sum : OUT STD_LOGIC_VECTOR(3 DOWNTO 0)
48          );
49  END COMPONENT;
50  BEGIN
51          i1 : fadd4
52          PORT MAP (
53  -- list connections between master ports and signals
54          a => a,
55          b => b,
56          ci => ci,
57          co => co,
58          sum => sum
59          );
60  init : PROCESS
61  -- variable declarations
62  BEGIN
63          -- code that executes only once
64          a<="0000";
65          b<="0000";
66          ci<='0';
67          for k in 0 to 1 loop
68              for j in 0 to 3 loop
69                  for i in 0 to 3 loop
70                      wait for 10 ns;
71                          a<=a+1;
72                  end loop;
73                  b<=b+1;
74              end loop;
75            ci<= not ci;
76          end loop;
77  WAIT;
78  END PROCESS init;
79  always : PROCESS
80  -- optional sensitivity list
81  -- (        )
82  -- variable declarations
83  BEGIN
84          -- code executes for every event on sensitivity list
85  WAIT;
86  END PROCESS always;
87  END fadd4_arch;
```

Figure 3: testbench for 4-bit full adder

2.5 Design Compilation and Synthesis

(1) Compile all VHDL source files to check for syntax errors and generate compiled design files.

(2) Analyze the post-synthesis reports to verify correct logic implementation.

(3) Generate Resource Usage Reports to optimize area and timing performance.

(4) Create netlist files for simulation and FPGA implementation.

3 Experimental Results and Analysis

3.1 Simulation Waveforms

The following waveforms demonstrate the functional behavior of the 4-bit full adder during simulation:

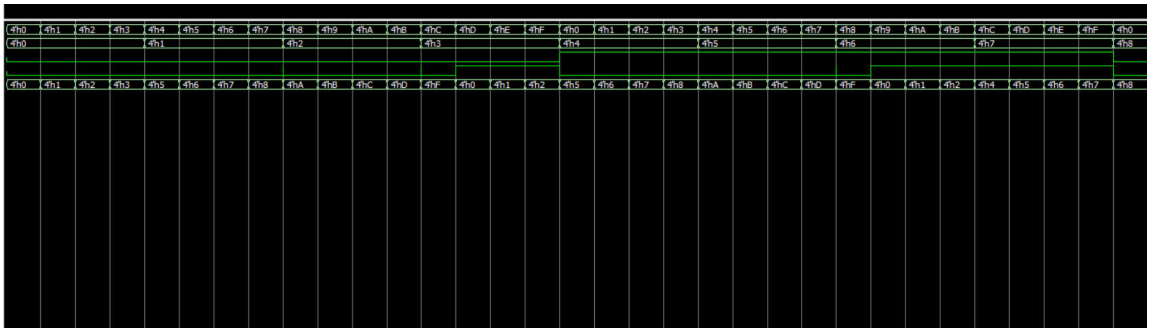

Figure 4: Simulation Waveforms 1

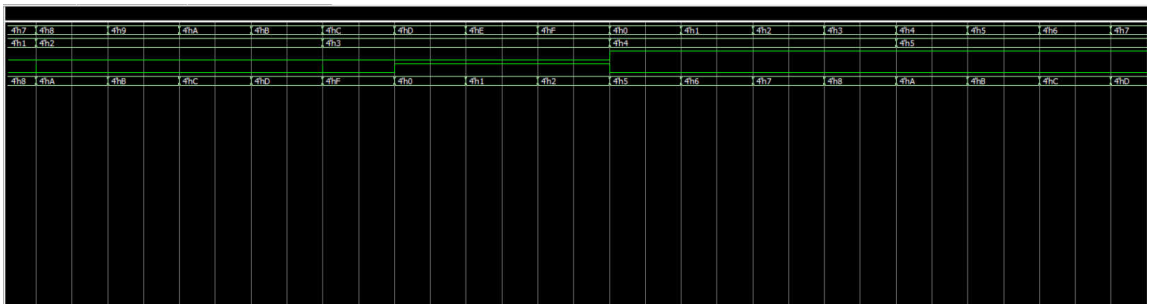

Figure 5: Simulation Waveforms 2(same as wave1 but zoomed in)

3.2 Conclusion

1. The simulation waveform successfully verifies the correctness of the 4-bit full adder, with the output values for both the sum and carry matching the expected results precisely. However, the simulation waveform displays some glitches. These glitches are likely due to the fact that "Enable Optimization" was not selected during the simulation. Without optimization, the simulation assumes an idealized model of the circuit, where all components react instantaneously. In reality, however, each unit or logic gate in the cir-

cuit has a propagation delay, meaning that the signal changes at slightly different times depending on the gate' s internal characteristics.

2. This delay, which varies across different logic elements in the design, can cause temporary inconsistencies or glitches in the output waveform. These glitches are artifacts of the simulation environment, which does not fully account for the real-world behavior of the circuit, such as signal timing and the delays inherent in physical components. Enabling the optimization feature would make the simulation more reflective of the actual circuit behavior, reducing these glitches by better modeling the propagation delay and the timing characteristics of the logic gates involved.